Robotic Navigation and Exploration

HW1: Path Planning

Name：顏維廷
Student id：M11452023

A* Implementation

- **Priority Queue Management**：Uses heapq to always expand the node with the lowest estimated total cost $F(v) = g(v) + h(v)$

```
current_f, current_node = heapq.heappop(self.open_list)
```

A* Implementation

- **Neighbor Expansion & Cost Updating**：Calls **get_neighbor_nodes** and updates the **g_score** if a shorter path is found

```
for neighbor in self.get_neighbor_nodes(current_node):
        if neighbor in self.visited_nodes:
            continue

        # Calculate tentative g_score
        tentative_g_score = self.g_score[current_node] + calculate_node_distance(current_node, neighbor)

        if neighbor not in self.enqueued_nodes or tentative_g_score < self.g_score.get(neighbor, float('inf')):

            # Update the best path to neighbor
            neighbor.parent = current_node
            neighbor.cost = tentative_g_score
            self.g_score[neighbor] = tentative_g_score
            f_score = tentative_g_score + calculate_node_distance(neighbor, self.goal_node)
            self.enqueued_nodes[neighbor] = tentative_g_score
            heapq.heappush(self.open_list, (f_score, neighbor))
```

RRT* Implementation

- **Choose Best Parent**：Searches within **search_radius** to find a neighbor that minimizes the cumulative cost of the **new_node**

```
for near_node in near_nodes:
    if check_collision_free(self.occupancy_map, near_node, new_node):
        cost = near_node.cost + calculate_node_distance(near_node, new_node)
        if cost < min_cost:
            best_parent = near_node
            min_cost = cost

    new_node.parent = best_parent
    new_node.cost = min_cost
```

RRT* Implementation

- **Rewiring the Tree**：Checks if neighboring nodes can achieve a lower cost by switching their **parent** to the **new_node**

```
for near_node in near_nodes:
    if near_node == best_parent: continue
    new_potential_cost = new_node.cost + calculate_node_distance(new_node, near_node)
    if new_potential_cost < near_node.cost:
          if check_collision_free(self.occupancy_map, new_node, near_node):
             near_node.parent = new_node
             near_node.cost = new_potential_cost
```

RRT* Implementation

- **Goal Threshold Check**：Sets **is_done** when the tree reaches within **goal_threshold** of the target coordinates

```
if calculate_node_distance(new_node, self.goal_node) <= self.goal_threshold:
    if check_collision_free(self.occupancy_map, new_node, self.goal_node):
        self.goal_node.parent = new_node
        self.goal_node.cost = new_node.cost + calculate_node_distance(new_node, self.goal_node)
        self.visited_nodes.add(self.goal_node)
        self.is_done.set()
```